

Aluno: Matheus Cardoso Pereira Santos **Registro Acadêmico:** a2609380

Disciplina: Teste de Software

Professor: Bernardo Lima

Prática de Laboratório 01

1) (0,5) Elabore um fichamento do Capítulo 1 do livro Introdução ao Teste de Software, disponível no moodle. A ficha deve conter, no mínimo:

a. Um resumo do texto, apresentando o contexto, os objetivos do capítulo, e seus conteúdos principais (deve conter em volta de 600 palavras)

b. Um conjunto de citações diretas indicando o conteúdo mais relevante do texto

c. Uma lista dos pontos principais de cada seção

Resposta:

FICHAMENTO TEXTUAL – LIVRO 01
DELAMARO, Márcio Eduardo; MALDONADO, José Carlos; JINO, Mario. Introdução ao Teste de Software . Rio de Janeiro: Elsevier, 2007.
Resumo do texto: O primeiro capítulo tem como objetivo estabelecer a base conceitual e a terminologia que será utilizada ao longo de toda a obra. Os autores (Delamaro, Maldonado e Jino) iniciam justificando a necessidade de uma linguagem comum, pois termos técnicos da área de teste de software adquirem significados específicos, distintos do uso coloquial. O contexto apresentado é o da crescente complexidade no desenvolvimento de software, na qual a atividade humana é apontada como a principal responsável pelos erros existentes. Para mitigar esse problema, os autores introduzem o conceito de Validação, Verificação e Teste (VV&T) como um conjunto de atividades destinadas a garantir a conformidade do software com o que será necessário. Nesse ponto, fazem uma distinção fundamental entre atividades estáticas (como revisões e <i>walkthroughs</i>), que não exigem a execução do programa, e atividades dinâmicas, nas quais o teste de software se enquadra, por basear-se na execução do programa com entradas específicas. Em seguida, o capítulo aprofunda a nomenclatura técnica. Define-se defeito como uma condição estática no código (um passo, processo ou definição de dados incorreto), resultante de um engano humano. Quando um programa com defeito é executado, há a possibilidade de que ocorra um erro, caracterizado por um estado inconsistente do programa. Esse erro, por sua vez, pode levar a uma falha, que é a manifestação externa de um comportamento inesperado ou resultado incorreto. Também são formalmente definidos domínio de entrada, dado de teste e caso de teste (dado de teste + resultado esperado), além do papel do oráculo, que decide sobre a correção do resultado obtido. As fases da atividade de teste são alinhadas ao ciclo de desenvolvimento. O teste de unidade foca nos menores componentes (funções, métodos, classes) para identificar erros locais de algoritmo ou lógica.

O teste de integração verifica a interação entre essas unidades à medida que são combinadas. O teste de sistema avalia o produto completo em relação aos requisitos funcionais e não funcionais. Destaca-se também o teste de regressão, realizado após modificações no software para garantir que alterações não introduziram novos defeitos e que as funcionalidades preexistentes permanecem válidas.

Para lidar com a impossibilidade do teste exaustivo (executar todas as entradas possíveis), o capítulo introduz os critérios de teste. A ideia central é particionar o domínio de entrada em subdomínios de dados semelhantes e selecionar representativos de cada um. Os critérios de teste são as regras que definem como particionar o domínio — podendo ser funcionais (baseados na especificação), estruturais (baseados no código) ou baseados em defeitos. Um conjunto de casos de teste que atende a todos os requisitos de um critério é chamado de adequado a esse critério.

Por fim, o capítulo discute as limitações teóricas e o verdadeiro objetivo do teste. Devido à infactibilidade do teste exaustivo e a questões indecidíveis (como saber se um caminho no código é executável), o teste não pode provar a ausência de defeitos. Seu objetivo principal é, portanto, mostrar a presença de defeitos quando eles existem, aumentando a confiança no software. O capítulo conclui com uma breve introdução a temas como depuração (localizar e corrigir defeitos), automação de teste e análise de confiabilidade, que serão detalhados em capítulos posteriores.

Citações diretas:

- “Definimos “defeito” (do inglês, *fault*) como sendo um passo, processo ou definição de dados incorretos e “engano” (*mistake*) como a ação humana que produz um defeito. Assim, esses dois conceitos são estáticos, pois estão associados a um determinado programa ou modelo e não dependem de uma execução particular”. (DELAMARO; MALDONADO; JINO, 2007, p. 02)
- “A atividade de teste é complexa. São diversos os fatores que podem colaborar para a ocorrência de erros. Por exemplo, a utilização de um algoritmo incorreto para computar o valor das mensalidades a serem pagas para um empréstimo ou a não-utilização de uma política de segurança em alguma funcionalidade do software são dois tipos distintos de engano e, de certa forma, encontram-se em níveis diferentes de abstração.” (DELAMARO; MALDONADO; JINO, 2007, p. 03)
- “Por isso, a atividade de teste é dividida em fases com objetivos distintos. De uma forma geral, podemos estabelecer como fases o teste de unidade, o teste de integração e o teste de sistemas. O teste de unidade tem como foco as menores unidades de um programa, que podem ser funções, procedimentos, métodos ou classes. Nesse contexto, espera-se que sejam identificados erros relacionados a algoritmos incorretos ou mal implementados, estruturas de dados incorretas, ou simples erros de programação.” (DELAMARO; MALDONADO; JINO, 2007, p. 04)
- “No teste de integração, que deve ser realizado após serem testadas as unidades individualmente, a ênfase é dada na construção da estrutura do sistema. À medida que as diversas partes do software são colocadas para trabalhar juntas, é preciso verificar se a interação entre elas funciona de maneira adequada e não leva a erros.” (DELAMARO;

MALDONADO; JINO, 2007, p. 04)

- “Depois que se tem o sistema completo, com todas as suas partes integradas, inicia-se o teste de sistema. O objetivo é verificar se as funcionalidades especificadas nos documentos de requisitos estão todas corretamente implementadas. Aspectos de correção, completude e coerência devem ser explorados, bem como requisitos não funcionais como segurança, performance e robustez. Muitas organizações adotam a estratégia de designar uma equipe independente para realizar o teste de sistemas.” (DELAMARO; MALDONADO; JINO, 2007, p. 04)
- Por isso costumamos dizer que o objetivo da atividade de teste não é, como podem pensar muitos, mostrar que um programa está correto. Ao invés disso, o objetivo é mostrar a presença de defeitos caso eles existam. (DELAMARO; MALDONADO; JINO, 2007, p. 06)
- “Muitos confundem, talvez com certa razão, depuração e teste de software. Embora fortemente relacionadas, são duas atividades diversas. O teste de software tem como objetivo, como vimos anteriormente, mostrar que o software não funciona como o esperado, ou seja, possui defeitos. A depuração, por sua vez, deve localizar as causas que levaram o software a falhar e indicar como ele deve ser corrigido.” (DELAMARO; MALDONADO; JINO, 2007, p. 06)
- “Podemos definir confiabilidade como a tendência de um software em operar de maneira correta. Certamente essa é uma das mais importantes – se não a mais importante – características desejadas de um software. Assim, a análise de confiabilidade de software, ou seja, procurar medir de forma objetiva essa característica tornou-se um fator de extrema importância nas organizações que desenvolvem software de maneira profissional.” (DELAMARO; MALDONADO; JINO, 2007, p. 07)

Pontos principais de cada seção:

1.1 Introdução:

- Complexidade do software → erro humano como principal fonte.
- Necessidade de linguagem comum.
- Introdução ao VV&T (Validação, Verificação e Teste).

1.2 Validação, verificação e teste de software:

- Estáticas: revisões, inspeções, walkthroughs (não executam o programa).
- Dinâmicas: teste de software (execução do programa com entradas específicas).

1.3 Alguns termos do jargão:

- Engano (mistake), defeito (fault), erro (error), falha (failure).
- Domínio de entrada, dado de teste, caso de teste, oráculo.

1.4 Fases da atividade de teste:

- Teste de unidade.
- Teste de integração.

- Teste de sistema.
- Teste de regressão.

1.5 Técnicas e critérios de teste:

- Estáticas: revisões, inspeções, walkthroughs (não executam o programa).
- Dinâmicas: teste de software (execução do programa com entradas específicas).

1.6 Características e limitações:

- Teste não prova ausência de defeitos (indecidibilidade).
- Objetivo: mostrar presença de defeitos e aumentar confiança.

1.7 Temas relacionados:

- Depuração (localização e correção).
- Automação de teste.
- Análise de confiabilidade.

FICHAMENTO TEXTUAL – LIVRO 02

KHORI KOV, Vladimir. **Unit Testing: Principles, Practices and Patterns**. Shelter Island: Manning Publications, 2020.

Resumo do texto:

O capítulo 1 do livro *Unit Testing: Principles, Practices and Patterns*, de Vladimir Khorikov, tem como objetivo estabelecer o propósito fundamental da atividade de teste de unidade e apresentar os principais conceitos que serão desenvolvidos ao longo da obra. O autor parte da constatação de que, embora o teste de unidade seja amplamente adotado na indústria de software, a discussão sobre o que constitui um bom teste ainda é incipiente. A questão deixou de ser "devemos escrever testes?" e passou a ser "o que significa escrever bons testes?".

Khorikov argumenta que o objetivo central do teste de unidade não é simplesmente alcançar alta cobertura de código ou seguir modismos técnicos, mas sim viabilizar o crescimento sustentável do projeto de software. O autor introduz o conceito de *entropia* aplicado ao software: sistemas tendem a se deteriorar ao longo do tempo, e cada modificação no código aumenta o nível de desordem. Sem uma prática consistente de testes, o projeto entra em uma fase de estagnação, onde o esforço necessário para implementar novas funcionalidades cresce exponencialmente enquanto o progresso efetivo diminui.

O autor diferencia projetos com bons testes, maus testes e nenhum teste. Projetos sem testes começam rápido, mas rapidamente entram em colapso. Projetos com testes mal escritos apresentam uma melhora inicial, mas inevitavelmente caem na mesma estagnação. Já projetos com testes bem elaborados mantêm a velocidade de desenvolvimento ao longo do tempo, permitindo refatorações seguras e adaptações rápidas às mudanças de requisitos.

Khorikov critica o uso de métricas de cobertura (como *code coverage* e *branch coverage*) como indicadores de qualidade. Ele demonstra que é possível atingir 100% de cobertura com testes inúteis (como testes sem asserções) ou que apenas exercitam o código sem verificar resultados. As métricas

de cobertura são úteis como indicador negativo — baixa cobertura sinaliza problema —, mas são péssimas como indicador positivo — alta cobertura não garante qualidade. O autor alerta ainda para o risco de transformar uma determinada porcentagem de cobertura em meta, pois isso cria incentivos perversos que desviam o foco do que realmente importa.

O capítulo também aborda as propriedades de um conjunto de testes bem-sucedido: (1) estar integrado ao ciclo de desenvolvimento (executado a cada alteração); (2) focar nas partes mais importantes do código (em geral, o modelo de domínio); e (3) fornecer máximo valor com mínimo custo de manutenção. Khorikov enfatiza que testes são código e, como tal, representam um passivo — cada linha de teste adicionada aumenta a superfície de manutenção. Portanto, é essencial manter apenas testes cujo valor líquido seja positivo e descartar os demais.

Por fim, o autor antecipa os temas que serão desenvolvidos ao longo do livro: diferenciação entre testes bons e ruins, refatoração de testes, estilos de teste, uso de testes de integração e identificação de antipadrões. A abordagem promete fornecer um referencial teórico sólido para analisar qualquer teste de unidade e tomar decisões conscientes sobre sua manutenção.

Citações diretas:

- “Learning unit testing doesn’t stop at mastering the technical bits of it, such as your favorite test framework, mocking library, and so on. There’s much more to unit testing than the act of writing tests. You always have to strive to achieve the best return on the time you invest in unit testing, minimizing the effort you put into tests and maximizing the benefits they provide. Achieving both things isn’t an easy task.” (KHORIKOV, 2020, p. 3)
- What is the goal of unit testing, then? The goal is to enable sustainable growth of the software project. The term sustainable is key. It’s quite easy to grow a project, especially when you start from scratch. It’s much harder to sustain this growth over time. (KHORIKOV, 2020, p. 6)
- “In software, entropy manifests in the form of code that tends to deteriorate. Each time you change something in a code base, the amount of disorder in it, or entropy, increases. If left without proper care, such as constant cleaning and refactoring, the system becomes increasingly complex and disorganized. Fixing one bug introduces more bugs, and modifying one part of the software breaks several others—it’s like a domino effect. Eventually, the code base becomes unreliable. And worst of all, it’s hard to bring it back to stability.” (KHORIKOV, 2020, p. 6)
- “At this point, I hope you can see that relying on coverage metrics to determine the quality of your test suite is not enough. It can also lead to dangerous territory if you start making a specific coverage number a target, be it 100%, 90%, or even a moderate 70%. The best way to view a coverage metric is as an indicator, not a goal in and of itself.” (KHORIKOV, 2020, p. 15)
- The only point in having automated tests is if you constantly use them. All tests should be integrated into the development cycle. Ideally, you should execute them on every code change, even the smallest one. (KHORIKOV, 2020, p. 16)

- “It’s important to direct your unit testing efforts to the most critical parts of the system and verify the others only briefly or indirectly. In most applications, the most important part is the part that contains business logic --- the domain model. Testing business logic gives you the best return on your time investment.” (KHORIKOV, 2020, p. 17)

Pontos principais de cada seção:

1.1 The current state of unit testing

- Unit testing is widely adopted and considered mandatory in most companies.
- The discussion has shifted from “should we write tests?” to “what does it mean to write good tests?”
- Many projects have lots of tests but still suffer from slow development, constant bugs, and high maintenance costs.
- There is little comprehensive material on what distinguishes good tests from bad ones.

1.2 The goal of unit testing

- The main goal is to enable **sustainable growth** of the software project.
- Without tests, development speed declines rapidly over time (software entropy).
- Good tests act as a safety net, preventing regressions and allowing safe refactoring.
- Bad tests can have zero or negative net value due to high maintenance costs and false alarms.

1.2.1 What makes a good or bad test?

- Not all tests are created equal; some provide value, others don’t.
- Bad tests raise false alarms, don’t catch regressions, are slow, and are hard to maintain.
- The net value of a test is its benefit minus its upkeep cost.

1.3 Using coverage metrics to measure test suite quality

- Coverage metrics (code coverage and branch coverage) are often used but have serious limitations.
- They can be gamed and don’t guarantee test quality.

1.3.1 Understanding the code coverage metric

- Measures the ratio of lines of code executed by tests.
- Easy to inflate by making code more compact; doesn’t reflect actual testing quality.

1.3.2 Understanding the branch coverage metric

- Measures the ratio of control structures (if, switch) traversed by tests.
- More precise than line coverage, but still insufficient.

1.3.3 Problems with coverage metrics

- **No guarantee of verification:** tests may exercise code without assertions.
- **No visibility into external libraries:** coverage doesn’t account for code paths inside third-party dependencies.

1.3.4 Aiming at a particular coverage number

- Setting a coverage target creates perverse incentives.

- Coverage is a good **negative indicator** (low coverage signals trouble), but a bad **positive indicator** (high coverage doesn't mean good tests).

1.4 What makes a successful test suite?

A successful test suite has three key attributes.

1.4.1 It's integrated into the development cycle

- Tests must be run on every code change, ideally as part of the build process.

1.4.2 It targets only the most important parts of your code base

- Focus on the **domain model** (business logic).
- Infrastructure, external dependencies, and glue code should receive secondary attention.

1.4.3 It provides maximum value with minimum maintenance costs

- Keep only tests with a positive net value.
- Eliminate tests that cost more than the benefit they provide.

1.5 What you will learn in this book

- A frame of reference to analyze any test and distinguish good from bad.
- How to refactor tests alongside production code.
- Different styles of unit testing.
- Use of integration tests and end-to-end tests.
- Common anti-patterns in unit testing.

Summary

- Code tends to deteriorate (entropy); tests help counteract this.
- The goal of unit testing is sustainable project growth.
- Not all tests are valuable; focus on high-quality tests with positive net value.
- Coverage metrics are useful as negative indicators but not as quality guarantees.
- A successful test suite is integrated, targets important code, and balances value against maintenance cost.

Linguagem escolhida: Java

Framework padrão: JUnit 5 (JUnit Jupiter). É adotado pois possui uma API moderna, suporte a lambdas, e tem boa integração ao Maven, Gradle e IDEs bem utilizadas como IntelliJ e Eclipse.

Configuração das Dependências (Maven):

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.12.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Funções de Exemplo para Teste:

Uma classe utilitária *Calculadora* com quatro funções para serem testadas:

```
package com.exemplo.utils;
```

```
public class Calculadora {
```

```
    // 1. Soma simples
```

```
    public int somar(int a, int b) {
```

```
        return a + b;
```

```
    }
```

```
    // 2. Divisão com tratamento de exceção
```

```
    public double dividir(int numerador, int denominador) {
```

```
        if (denominador == 0) {
```

```
            throw new IllegalArgumentException("Denominador não pode ser zero");
```

```
        }
```

```
        return (double) numerador / denominador;
```

```
    }
```

```
    // 3. Verifica se um número é par
```

```
    public boolean isPar(int numero) {
```

```
        return numero % 2 == 0;
```

```
    }
```

```
    // 4. Calcula fatorial (para testar múltiplos casos)
```

```
    public int fatorial(int n) {
```

```
        if (n < 0) {
```

```
            throw new IllegalArgumentException("Número não pode ser negativo");
```

```
        }
```

```
        if (n == 0 || n == 1) {
```

```
            return 1;
```

```
        }
```

```
        return n * fatorial(n - 1);
```

```
    }
```

```
}
```

Classe de Teste com JUnit 5:

```
package com.exemplo.utils;
```

```
import org.junit.jupiter.api.*;
```

```
import org.junit.jupiter.params.ParameterizedTest;
```

```
import org.junit.jupiter.params.provider.CsvSource;
```

```
import org.junit.jupiter.params.provider.ValueSource;
```



```
import static org.junit.jupiter.api.Assertions.*;
import static java.time.Duration.ofMillis;

class CalculadoraTest {

    private Calculadora calculadora;

    @BeforeAll
    static void setupAll() {
        System.out.println("Iniciando testes da Calculadora...");
    }

    @BeforeEach
    void setup() {
        calculadora = new Calculadora();
        System.out.println("Instanciando calculadora para um teste");
    }

    @AfterEach
    void tearDown() {
        System.out.println("Teste finalizado");
    }

    @AfterAll
    static void tearDownAll() {
        System.out.println("Todos os testes da Calculadora finalizados");
    }

    // Teste 1: Soma - teste simples
    @Test
    @DisplayName("Teste de soma de dois números positivos")
    void testSomar() {
        int resultado = calculadora.somar(3, 5);
        assertEquals(8, resultado, "3 + 5 deve ser 8");
        assertNotEquals(10, resultado);
    }

    // Teste 2: Divisão com exceção e uso de delta
    @Test
    @DisplayName("Teste de divisão com sucesso e com exceção")
    void testDividir() {
```

```

// Teste de sucesso
double resultado = calculadora.dividir(10, 4);
assertEquals(2.5, resultado, 0.001, "10/4 deve ser 2.5");

// Teste de exceção - verifica se a exceção correta é lançada
Exception exception = assertThrows(IllegalArgumentException.class, () -> {
    calculadora.dividir(5, 0);
});
assertEquals("Denominador não pode ser zero", exception.getMessage());
}

// Teste 3: Parâmetro - usando ParameterizedTest
@ParameterizedTest
@ValueSource(ints = {2, 4, 6, 8, 10})
@DisplayName("Teste de números pares com múltiplas entradas")
void testIsParComNumerosPares(int numero) {
    assertTrue(calculadora.isPar(numero), numero + " deveria ser par");
}

@ParameterizedTest
@ValueSource(ints = {1, 3, 5, 7, 9})
@DisplayName("Teste de números ímpares com múltiplas entradas")
void testIsParComNumerosImpares(int numero) {
    assertFalse(calculadora.isPar(numero), numero + " deveria ser ímpar");
}

// Teste 4: Fatorial com múltiplos casos usando CsvSource
@ParameterizedTest
@CsvSource({
    "0, 1",
    "1, 1",
    "2, 2",
    "3, 6",
    "4, 24",
    "5, 120"
})
@DisplayName("Teste de fatorial com diferentes valores")
void testFatorial(int entrada, int esperado) {
    assertEquals(esperado, calculadora.fatorial(entrada));
}

@Test

```

```
@DisplayName("Teste de fatorial com valor negativo")
void testFatorialNegativo() {
    assertThrows(IllegalArgumentException.class, () -> {
        calculadora.fatorial(-1);
    });
}

// Teste 5: Agrupando asserções (demonstração extra)
@Test
@DisplayName("Teste agrupado de múltiplas asserções")
void testAgrupado() {
    assertAll("Validações da calculadora",
        () -> assertEquals(5, calculadora.somar(2, 3)),
        () -> assertTrue(calculadora.isPar(4)),
        () -> assertFalse(calculadora.isPar(7)),
        () -> assertEquals(120, calculadora.fatorial(5))
    );
}

// Teste 6: Timeout (demonstração)
@Test
@DisplayName("Teste de timeout para operação longa")
void testTimeout() {
    assertTimeout(ofMillis(100), () -> {
        // Simula uma operação que deve ser rápida
        calculadora.fatorial(10);
    });
}
}
```